

Day 15 — Fairness, Normalization & Bias Reduction Engine

Design & Implementation Documentation

1. Objective

Day 15 improves fairness, reduces bias, and standardizes resume evaluation across the Zecpath pipeline. It is an additive layer that sits between the Day 13 ATS Scoring Engine / Day 14 Candidate Ranking Engine and any human decision-maker, and does not modify the schema or internal behavior of any prior day's module.

Five concrete tasks were scoped for this day, each mapped to a dedicated, independently-tested module:

Module	Responsibility	Fairness Task Addressed
normalizer.py	Standardizes heterogeneous candidate profiles into one canonical schema	Normalize resumes to standard format
masker.py	Redacts/neutralizes non-essential personal attributes before scoring	Mask non-essential personal attributes
keyword_balancer.py	Dampens and caps the influence of raw keyword matching on final score	Reduce over-dependence on keywords
score_normalizer.py	Rescales final scores across a candidate pool (z-score, percentile, robust)	Implement scoring normalization
bias_auditor.py	Applies the four-fifths rule to optional, out-of-band demographic labels	Evaluate bias indicators

2. Normalizing Resumes to a Standard Format

Two candidates with identical qualifications can look very different to a scorer purely because of resume formatting choices — a two-column layout vs. a single column, a skills table vs. a skills paragraph, "5 yrs" vs. "five years of experience". None of that variation is job-relevant signal, but left unaddressed it can quietly influence downstream scoring.

The ResumeNormalizer converts any raw candidate record (a merge of Day 5-11 outputs, or an equivalent dict) into a CanonicalCandidateProfile with:

- Standardized skills: lower-cased, whitespace-normalized, de-duplicated, and flattened out of category groupings (e.g. "Languages": [...], "Cloud": [...]) into one flat list.
- A unified degree ladder: PhD / Masters / Bachelors / Associate / Other / Unknown, absorbing dozens of real-world spelling variants (M.Tech, MBA, B.E., BSc, ...).
- Consistent experience units: years as a float, derived from an explicit field when present, summed from experience-record durations otherwise, or parsed from free text as a last resort.

The normalizer is alias-tolerant by design, consistent with the project's existing convention (Day 7 metadata envelope, Day 9-11 loaders) of absorbing schema drift between pipeline stages without requiring upstream modules to change.

3. Reducing Over-Dependence on Keywords

A candidate who pads a resume with every keyword from the job description can out-score a genuinely stronger candidate if the keyword-match component of the Day 13 scoring engine is left uncapped. Keyword count is a weak, easily gamed proxy for fit; semantic similarity (Day 12) and experience relevance (Day 10) are harder to game and more predictive of actual performance.

The KeywordDependencyReducer addresses this in two complementary ways:

- Diminishing returns: matches beyond a configurable threshold (default 8) are compressed with a square-root curve, so the 20th matched keyword contributes far less than the 5th.
- A weight cap: the keyword-match component may not exceed a configurable share of total weight (default 35%). Any excess weight is redistributed proportionally across the remaining Day 13 scoring components rather than discarded, so the total weight always still sums to 1.0.

This operates directly on the {component_name: (raw_score, weight)} shape already produced by Day 13's WeightProfile, so it can be inserted into the pipeline without altering the Day 13 or Day 14 schemas.

4. Scoring Normalization

Different roles produce final scores with different distributions — one role's candidates might cluster tightly around 0.6-0.7, while another's spread from 0.2 to 0.9. Without pool-level normalization, a raw score of 0.7 can mean "top of the pool" for one role and "middling" for another, which quietly distorts cross-role fairness (for example, the auto-reject threshold used by Day 14's zone-based ranking).

The ScoreNormalizer computes three complementary views for every candidate in a pool:

- z-score: standard deviations from the pool mean (population statistics, since the pool is the full population being compared, not a sample).
- Percentile rank: fraction of the pool scoring at or below this candidate.
- Robust normalized score: a median/IQR-centered, logistic-squashed rescale into [0, 1] that resists distortion from one or two extreme outlier scores.

Edge cases are handled explicitly rather than left to produce misleading output: an empty pool returns no records, a single-candidate pool returns the raw score unchanged for every view, and a zero-variance pool (all identical scores) reports a neutral z-score of 0.0 and percentile of 0.5 for everyone instead of dividing by zero.

5. Masking Non-Essential Personal Attributes

The AttributeMasker implements a "blind hiring" principle: a scorer cannot be biased by information it never sees. Personal fields that carry no job-relevant signal are redacted before a profile is exposed to any scoring stage.

Masked (never reaches scoring)	Preserved (job-relevant)
Name, gender/sex, date of birth, age, photo, marital status, nationality, religion, ethnicity/race	Skills, certifications, years of experience, job titles/seniority, degree tier
Street-level address (city/region retained)	City/region, when genuinely relevant to on-site requirements
Specific institution name (default: reduced to a neutral placeholder)	Degree tier (Bachelors / Masters / PhD / Associate / Other)
Gendered pronouns and honorifics in free text (he/she -> they, Mr./Ms. removed)	The substance of the sentence itself

A reversible identity mapping is retained separately for legitimate recruiter use (e.g. eventually contacting the candidate), but is explicitly excluded from the default masked-profile output and from every stored fairness report — verified directly by an automated test (`test_masker_reversible_map_not_in_default_dict_output`).

Institution-name masking is a deliberate default, not a hard rule: some roles genuinely require verifying a specific accreditation, so it can be turned off per invocation. It defaults to on everywhere else, because institution prestige correlates with socioeconomic background more than with job performance for most roles — a well-documented pathway for pedigree and class bias.

6. Evaluating Bias Indicators

The BiasAuditor never infers a candidate's protected characteristics from resume content — doing so would itself be a bias vector. It only evaluates bias when group labels are supplied separately and out-of-band (for example, from an anonymous post-decision compliance survey), and those labels are used exclusively for auditing, never fed back into scoring.

For each supplied dimension (e.g. gender, age band), the auditor computes:

- Per-group selection rate: shortlisted count divided by group size.
- The four-fifths rule: a standard adverse-impact screening heuristic (EEOC Uniform Guidelines) that flags a potential concern when the lowest group selection rate falls below 80% of the highest.
- Per-group score distribution: mean and median, for additional context beyond the binary selection-rate check.

Two guardrails prevent the auditor from overstating its conclusions: when no group labels are supplied at all, or when every group is too small to be statistically meaningful (fewer than 5 members, by default), the report is marked "insufficient data" and says so in its narrative rather than emitting a false positive or false negative. Every report explicitly states that the four-fifths screen is a heuristic, not a legal or causal determination.

7. Pipeline Integration

The FairnessBiasEngine ties all five components into a single call, `process_candidate_pool()`, which:

- Normalizes each raw profile into a CanonicalCandidateProfile.
- Masks each raw profile, independently of normalization, so both a clean canonical view and a safe-to-score masked view are always produced together.
- Rebalances each candidate's supplied Day 13 score components and computes an adjusted final score.
- Normalizes the resulting adjusted scores across the full candidate pool for the role.
- Runs the bias audit across any supplied demographic dimensions, using the adjusted scores and any supplied shortlist flags.

The result serializes to a single JSON report per role via FairnessResultStore, tagged with the project's standard `schema_version / model_version / pipeline_version` metadata envelope (Day 7 convention), so downstream tooling can consume it the same way it consumes every other day's output.

8. Testing & Quality

25 automated tests cover every module in isolation plus one full end-to-end integration test, executed via a timestamped test runner consistent with prior days' delivery pattern. Notable edge cases specifically covered:

- Unrecognized degree text falls back to "Other" with an explanatory note rather than silently misclassifying it.

- A candidate with no supplied score components still produces a valid (zero) adjusted score rather than raising an exception.
- Zero-variance and single-candidate score pools are normalized without division-by-zero errors.
- Tiny demographic groups are explicitly flagged as low-confidence and excluded from a four-fifths conclusion.

All 25 tests pass. The suite, fixtures, and a timestamped log are included in the deliverable archive under tests/.

9. Known Limitations & Design Choices

- No demographic inference: group labels must come from a deliberate, separate process; the engine will not guess them.
- The four-fifths rule is a screening heuristic only, and the auditor is explicit about that in every generated narrative.
- Institution-name masking and keyword-share caps are both configurable per invocation, defaulting to the fairness-preserving setting.
- This module assumes Day 13's component shape (`{component_name: (raw_score, weight)}`); if that shape changes upstream, only the `keyword_balancer`'s alias list needs updating.